FINAL MANUAL

JavaScript and React

From Browser Mechanics to Predictable UI Systems

A dense operational guide for strong programmers who want to stop treating the UI layer as magic.

Focus: what creates UI, updates UI, removes UI, and explains why those changes happen.

Document Promise

This manual compresses the highest-leverage 20 percent of browser JavaScript and React into one continuous system. The emphasis is not on memorizing APIs. The emphasis is on tracing cause and effect from code to DOM mutation to rendered pixels.

The intended learner already knows programming. Therefore the explanations assume comfort with functions, objects, references, control flow, and debugging, but they do not assume prior fluency with browser runtime behavior.

Table of Contents

Control Frame

Already Covered

Your programming background already covers general computational reasoning: variables, references, functions, data structures, branching, loops, decomposition, and debugging discipline.

The broader curriculum that this final manual compresses includes HTML and CSS structure, DOM and rendering basics, JavaScript internals, JavaScript patterns that matter for React, and React internals. This document will restate only the parts that directly govern real UI work.

This Document Covers

The minimum HTML and CSS model required to understand what JavaScript can control.

DOM selection, creation, insertion, removal, mutation, and the event system.

Time, timers, repeated behavior, and why screen updates happen when they do.

State as a source of truth in plain JavaScript, then the React model that formalizes that idea.

React render and update flow, file execution flow, practical mini systems, and the mistakes that repeatedly block new web developers.

Out of Scope

Exhaustive CSS property catalogs, animation libraries, routing ecosystems, advanced bundler internals, server rendering architecture, state libraries, full testing strategy, and framework-specific deployment workflows.

Those topics matter later, but they are not part of the foundational 20 percent that controls whether you can reason about interactive screens confidently today.

Why This 20 Percent Is Enough To Start Building Real Systems

Most freelance and junior-to-mid product work reduces to the same loop: receive input, hold state, map state to visible UI, react to time, and update or remove DOM nodes predictably.

If you can trace that loop precisely, you can build forms, lists, dashboards, filters, menus, dialogs, search boxes, counters, and CRUD interfaces without waiting for tutorials. Everything else becomes an extension, not a mystery.

1. Minimal HTML and CSS Mental Model

HTML is not the screen. HTML is a text format that the browser parses into a tree of node objects called the DOM. The browser later combines that DOM tree with CSS rules to compute layout and paint pixels. Therefore JavaScript does not control pixels directly in normal UI work. It controls DOM nodes and style-related properties, and the browser translates those changes into visual results.

This distinction matters because many beginner confusions come from collapsing three layers into one: document structure, style calculation, and actual painting. When you change a node with `textContent`, you are modifying DOM state. When that change affects what should be visible, the browser eventually recalculates layout and repaints. The screen is a consequence, not the primary data structure.

A useful mental model is: HTML gives you addresses, CSS decides how those addresses look, and JavaScript decides when the addresses are added, removed, or changed.

1.1 The Small Set of Tags That Matter Most

Element	Why it matters	Typical JavaScript control point
div	Generic structural container. Most application layout is built from grouped boxes.	Insert children, toggle classes, attach delegated listeners.
button	Primary action trigger. It is already keyboard-aware and semantically clickable.	Listen for clicks, disable while processing, update label.
input	Single-value user entry point.	Read and write value, respond to input or change.
form	Submission boundary that packages user intent.	Handle submit, prevent default navigation, serialize values.
ul / li	Natural representation for dynamic collections.	Append items, remove items, reorder items, delegate events.
label	Connects explanatory text to inputs and improves click target behavior.	Use with form fields and accessibility flow.
img	Visual asset node whose display depends on src and loading state.	Swap src, lazy-load, show placeholder or error state.
a	Navigation trigger.	Read href, intercept for SPA routing, style visited or active state.
section / header / main / footer	Structural anchors that make the document readable and easier to inspect.	Useful for selection, grouping, and maintainable DOM shape.

1.2 High-Utility Attributes

Attribute	Operational use
id	Stable direct lookup for one node. Useful when the node is unique and central.
class	Style grouping and batch selection. Useful when multiple nodes share behavior or appearance.
value	Live input state for text-like controls.
disabled	Blocks user interaction. Important for loading and validation states.
checked	Boolean UI state for checkbox and radio inputs.
data-*	Attaches metadata that JavaScript can read without abusing classes or text.

style	Fast direct visual mutation. Fine for small cases, but harder to scale than classes.
src / href	Control media loading and navigation destination.

1.3 The CSS Slice You Actually Need

- CSS controls visibility, size, position, spacing, overflow, and transitions. JavaScript often changes class names or inline styles, but CSS defines what those changes mean visually.
- If an element exists in the DOM but is hidden, the bug is usually in CSS or layout, not in element creation.
- If the DOM looks correct in DevTools but the screen looks wrong, inspect display, position, width, height, overflow, opacity, and z-index before assuming JavaScript failed.

Operational Rule

The browser can only paint what the DOM and computed styles describe. Therefore debugging UI should always ask two separate questions: did the node exist, and if it existed, why was it not visible in the expected place?

Exercise - DOM Visibility Check

Create a banner element in HTML and hide it with CSS. Then show it with JavaScript after a button click. Diagnose the difference between a node that does not exist and a node that exists but is visually hidden.

Algorithmic direction: Use DevTools to inspect both DOM presence and computed style. Force yourself to prove which layer is failing before changing code.

2. DOM Control - The Core Power Layer

The DOM is a mutable tree of node objects. Each element knows its parent, its children, its attributes, and eventually its place in layout. When you write DOM code, you are editing that tree. The browser is not executing intent. It is obeying concrete mutations to that object graph.

Expert-level DOM work is not about memorizing many methods. It is about understanding the lifecycle of a node: find an anchor, create a node in memory, configure its content and metadata, insert it into the tree, update it when state changes, and remove it when it is no longer needed.

2.1 Selection Means Finding Stable Anchors

Selection is not a cosmetic step. It defines what part of the document your code owns. `document.getElementById` is best when one unique node is central to the interaction. `querySelector` is better when the address is expressed structurally, such as a class, data attribute, or descendant relationship.

```javascript
const list = document.getElementById('notifications');
const saveButton = document.querySelector('[data-action="save"]');

saveButton.addEventListener('click', () => {
  console.log('These references are stable anchors for later work.');
```

```
});
```

2.2 Creation, Insertion, Removal, and Mutation

```javascript
const list = document.getElementById('notifications');

const item = document.createElement('li');
item.textContent = 'Profile saved';
item.dataset.kind = 'success';

list.appendChild(item);
```

`createElement` allocates a node in memory, but that node is not yet part of the visible document. `textContent` attaches plain text to the node safely. Setting `dataset.kind` stores metadata on the element. Only `appendChild` changes the tree shape by placing the node under `list`. Therefore the visible change begins only at insertion.

- `appendChild` inserts at the end of a parent.
- `insertBefore` inserts relative to an existing child.
- `remove` deletes a node from its parent.
- `removeChild` removes a known child through its parent. Useful when ownership is explicit.
- `setAttribute` updates serialized attributes. Useful for data, accessibility, and general DOM metadata.
- `textContent` replaces text without reparsing HTML. Prefer this when you mean text, not markup.
- `innerHTML` reparses markup and replaces descendants. Powerful, but easy to misuse because it destroys and recreates subtree nodes.

2.3 Cause and Effect Map

Operation	Mechanical result	What the user sees
Select element	You obtain a reference to an existing node object.	Nothing changes yet.
Create node	A node exists in memory only.	Nothing changes yet.
Insert node	Parent-child relationship changes in the live DOM.	A new element may appear after layout and paint.
Mutate text or attributes	Node content or metadata changes.	Text, attributes, or style-driven appearance may change.
Remove node	The node leaves the live tree.	The element disappears after the next paint.

2.4 The Important Warning About innerHTML

When you assign to `innerHTML`, the browser parses the string as markup and replaces the subtree under that element. That means existing child nodes are discarded. Event listeners attached directly to those child nodes are lost because the original nodes no longer exist.

Therefore `innerHTML` is reasonable for controlled template insertion, but it is a poor default update mechanism for interactive trees. If you need incremental updates, create and update nodes directly or let React manage the subtree.

Exercise - Notification Feed

Build a feed where clicking a button adds a new item with a timestamp. Then add a second button that removes the oldest item. Implement it once with direct node creation and once with innerHTML string replacement.

Algorithmic direction: Observe which version preserves listeners and state more predictably. Compare not just syntax length, but update semantics.

3. Events - The User Interaction System

An event is the browser telling your program that something happened: a click, a key press, a text change, a form submission, a focus change, or many others. The event system matters because most UI programs are dormant until something external triggers code.

The central pattern is `addEventListener`. You register a function on a node, and the browser calls that function when a matching event flows through the tree. The event object describes what happened, where it happened, and what browser-managed default action may follow.

3.1 Capture, Target, Bubble

Phase	What it means	Why you care
Capture	The event moves from outer ancestors toward the target.	Useful when a parent wants first access before the target handles it.
Target	The event reaches the exact node where it originated.	This is the node the user directly interacted with.
Bubble	The event moves back up through ancestors.	Most application code relies on this because parents can observe child interactions.

Most application code uses bubbling because it supports event delegation. Instead of attaching one listener to every child, you attach one listener to a stable parent and inspect `event.target` or `closest` to determine which child caused the event.

```javascript
const list = document.getElementById('tasks');

list.addEventListener('click', (event) => {
  const removeButton = event.target.closest('[data-remove]');
  if (!removeButton) return;

  const item = removeButton.closest('li');
  if (item) item.remove();
});
```

Delegation matters because children may be added later. If your listener lives on the parent, future children still work without extra binding. This is one of the highest-leverage patterns in non-React JavaScript because dynamic lists are everywhere.

3.2 Input, Keyboard, and Form Events

- Use input events when you need live value tracking as the user types.
- Use change events when the control should commit less frequently, depending on control type.
- Use keydown for keyboard shortcuts because it fires before the browser completes the key action.
- Use submit on forms because pressing Enter or clicking a submit button should hit the same logic path.

3.3 Default Actions and Prevention

Some events have browser-managed defaults. A form submit navigates. A link click changes location. A checkbox click toggles checked state. Calling `event.preventDefault()` says: keep the event, but skip the browser's built-in follow-up behavior.

Two Separate Controls

preventDefault: Stop the browser's default action.
stopPropagation: Stop the event from continuing through capture or bubble listeners.
These are not interchangeable. Many bugs come from using propagation control when the real need was to suppress navigation or submission.

Exercise - Delegated List Actions

Create a list where one delegated listener handles item removal, item completion toggling, and opening an edit mode based on different data attributes on buttons.
Algorithmic direction: Treat the parent as the stable event boundary. Branch by closest matching control, not by hard-coded child positions.

4. Time and Behavior

UI is not only spatial; it is temporal. Messages disappear after delays. Polling repeats. Loading indicators animate. Validation may be deferred. Therefore you need a mental model of when code runs, not just what code does.

The useful slice of the event loop for application work is simple: synchronous code runs first on the current call stack. Timers schedule future callbacks. After the current work and queued microtasks complete, the browser may recalculate style, layout, and paint. Therefore a visual update can be caused by synchronous DOM changes but still appear only after the browser gets a chance to render.

```
event loop sketch
click event -> handler runs now
state or DOM mutations happen now
microtasks finish before the browser moves on
browser may recalculate style and layout
browser paints
later timers or new events run
```

4.1 Timers

- `setTimeout` schedules one callback in the future.
- `setInterval` schedules repeated callbacks until cancelled.
- `clearTimeout` and `clearInterval` are part of correctness, not optional cleanup.

```javascript
const status = document.getElementById('status');
let remaining = 5;

const id = setInterval(() => {
  status.textContent = `Refreshing in ${remaining}...`;
  remaining -= 1;

  if (remaining < 0) {
    clearInterval(id);
    status.textContent = 'Refreshing now';
  }
}, 1000);
```

Repeated timers look simple but can drift because the browser does not guarantee exact millisecond precision. The important operational takeaway is not nanosecond accuracy. It is understanding that timers create independent future entry points into your program. If you forget cleanup, you create background behavior that continues after the UI conceptually moved on.

4.2 Animation and Rendering

If you are updating visual state every frame, `requestAnimationFrame` usually matches browser paint timing better than raw intervals. It is not required for most CRUD UI, but the concept is useful: some work should align with rendering cadence, not arbitrary time slices.

High-Leverage Timing Rule

When debugging time-based UI, always ask: who created this timer, what UI state does it read, and who is responsible for stopping it when that UI is gone?

Exercise - Auto-Hide Toast

Build a toast notification system where new notifications appear immediately and disappear after three seconds unless the user hovers over them.

Algorithmic direction: Separate creation, timer ownership, and removal. The challenge is not adding the element. The challenge is coordinating lifetime cleanly.

5. State and Change in Plain JavaScript

State is the smallest set of mutable data that determines what the UI should look like right now. That sentence is more useful than any framework-specific definition because it scales from raw DOM code to React and beyond.

A disciplined JavaScript UI keeps state in ordinary variables or objects, then derives the visible interface from that state. The moment you mix that with arbitrary manual DOM mutations spread across many handlers, you create hidden dependencies. Some handlers update the data. Some handlers update the DOM. Some do both. That is where maintainability collapses.

```javascript
const state = { count: 0 };
const valueEl = document.getElementById('count-value');

function render() {
  valueEl.textContent = String(state.count);
}

document.getElementById('inc').addEventListener('click', () => {
  state.count += 1;
  render();
});

render();
```

This pattern is primitive but important. The variable `state.count` is the source of truth. The DOM is a projection. Whenever state changes, `render` re-applies the visible representation. The weakness is that you must remember to call `render` from every mutation path. That memory burden grows linearly with features.

5.1 Manual State Synchronization Gets Expensive

Problem	Why it appears in plain DOM apps
Missed updates	One code path mutates data but forgets to update the affected nodes.
Duplicated logic	Multiple handlers reconstruct the same DOM updates independently.
Order bugs	Derived values are recomputed before all relevant state changes land.
Identity bugs	Nodes are recreated when they should be updated, or updated when they should be recreated.

The Foundational Rule

Do not let the DOM become the hidden primary state store unless that is explicitly your design. If application truth lives partly in JavaScript variables and partly in mutated nodes, debugging becomes archaeology.

Exercise - Single Source of Truth

Build a filterable list where an input box updates a state object, and a render function rebuilds the visible list from that state.

Algorithmic direction: Keep filtering logic in JavaScript state derivation, not in scattered direct DOM show or hide operations.

6. Why React Exists

React exists because imperative DOM code scales poorly when the amount of state and the number of UI dependencies both grow. The hard problem is not creating a button. The hard problem is guaranteeing that every visible part of the interface stays consistent with changing application data over time.

React changes the programming model. Instead of saying, 'When this state changes, mutate these five nodes in this order,' you say, 'Given current props and state, this is the UI description.' React then compares the new description to the previous one and performs the required DOM mutations.

Concern	Imperative DOM approach	React approach
Source of truth	Often split across variables and live nodes.	State and props describe UI; DOM is output.
Update strategy	Developer manually targets nodes.	Developer returns a new UI description; React reconciles.
Dynamic lists	Manual creation, update, removal, and listener management.	Render arrays of elements from data and let React track identity with keys.
Complexity growth	Mutation paths multiply across features.	Rendering stays centralized in component output.

This does not mean React eliminates complexity. Business logic is still your problem. But React removes a class of coordination bugs by making UI a function of state rather than a set of hand-authored mutation scripts.

React Is Not Magic

React is a runtime that stores state across renders, runs component functions to produce element descriptions, compares the new description with the previous tree, and commits DOM mutations. That is enough to explain most everyday behavior.

Exercise - Manual DOM vs Declarative UI

Take a small widget such as a searchable list. Write down every DOM mutation needed in an imperative version. Then write the state shape and render rule that a React version would use.

Algorithmic direction: The value is not the code. The value is seeing that React compresses many mutation branches into one render description.

7. React Core - Internal Logic, Not Surface Ritual

A React component is a function that maps input data to a UI description. JSX is only syntax sugar. It is compiled into calls that create element objects. Those element objects are not DOM nodes. They are plain JavaScript descriptions of what should exist.

```
jsx -> javascript
function SaveButton() {
  return <button className="primary">Save</button>;
}

// approximately becomes
function SaveButton() {
  return React.createElement(
```

```
    'button',
    { className: 'primary' },
    'Save'
  );
}
```

React receives the element tree returned by components, not rendered HTML text. Internally it keeps a record of what is currently mounted and what the next render wants. The reconciler compares those trees and determines whether a node should be inserted, updated, preserved, or removed.

7.1 Render Phase and Commit Phase

Stage	What React is doing	What is not happening yet
Render phase	Running component functions and producing the next element tree.	The DOM is not being mutated yet.
Reconciliation	Comparing next output to prior structure to compute required changes.	The browser still shows the old DOM.
Commit phase	Applying DOM mutations, refs, and effect setup or cleanup.	No further comparison is happening in this step.

This distinction explains a large percentage of React confusion. A re-render means React recalculated what the UI should be. It does not automatically mean the DOM changed. If the new output is effectively the same, the commit can be minimal.

7.2 How React Adds, Updates, and Removes UI

- If a node exists in the new tree but not the old tree, React inserts a corresponding DOM node during commit.
- If the same node identity remains but props or text change, React updates the existing DOM node.
- If a node exists in the old tree but not the new tree, React removes that DOM node during commit.
- If identity changes because type or key changed, React often tears down the old subtree and mounts a new one.

7.3 Identity and Keys

Keys are not a warning-suppression feature. Keys tell React which child across renders is conceptually the same entity. This matters because identity controls preservation of DOM nodes and component state.

```react
items.map((item) => (
  <li key={item.id}>
    {item.label}
  </li>
))
```

Identity Rule

Same type plus same position or key usually means preserve and update. Different type or different key means replace. Therefore keys are about continuity of meaning, not merely about list rendering syntax.

Exercise - Element Tree Diff

Represent two small UI trees as plain objects with type, props, key, and children. Determine which nodes are inserts, updates, preserves, and removals.

Algorithmic direction: Start with type and key first. Those two fields answer most identity questions before you even inspect props.

8. React State and Events

React state exists so values can survive across repeated calls to a component function. If components were only pure functions of props with no retained memory, interactive UI would be impossible. `useState` gives React-managed memory slots that are associated with a component across renders.

```react
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount((c) => c + 1)}>
      Count: {count}
    </button>
  );
}
```

The setter does not mutate the DOM directly. It enqueues an update for React. React later re-runs the component with a new state snapshot, produces a new element tree, and commits the necessary DOM changes. Therefore React updates are description-first, not mutation-first.

8.1 Snapshot Thinking

Each render sees its own snapshot of props and state. Event handlers created during that render close over that snapshot. This is why stale closure bugs happen: a future callback can still reference old values if you expected it to see newer renders automatically.

```react
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
  }

  return <button onClick={handleClick}>{count}</button>;
}
```

Both updates above use the same closed-over value of `count`. The functional form `setCount((c) => c + 1)` is safer for dependent updates because React can apply it against the latest queued value.

8.2 React Events vs DOM Events

- In plain DOM code, you attach listeners directly to nodes you own.

- In React, you describe handlers as props like onClick or onChange, and React wires them into its event system for that rendered tree.
- The practical takeaway is the same: a user event enters your code, your code updates state, React computes next UI, then DOM changes are committed.

8.3 Controlled Inputs

```react
function NameField() {
  const [name, setName] = useState('');

  return (
    <input
      value={name}
      onChange={(event) => setName(event.target.value)}
    />
  );
}
```

A controlled input is a good example of the React loop. The browser dispatches an input-related event. Your handler reads the latest text from the DOM event object. You store that value in React state. React re-renders, then writes the state value back into the input prop. The field stays synchronized because state is authoritative.

Correction Pattern For Stale State

Use functional updates when the next state depends on the previous state.

Use refs when a long-lived callback must read a current mutable value without forcing re-render.

Understand the reason, not only the fix: callbacks keep references to the render where they were created.

Exercise - Search Box

Build a component with an input, a list of items, and a derived filtered list that updates as the user types.

Algorithmic direction: Keep the raw query as state. Compute the filtered result during render instead of storing a second redundant state value unless there is a measured reason to cache it.

9. Rendering and Update Logic

When state changes, React does not ask, 'Which exact DOM node should I mutate first?' It asks, 'What should the next rendered output be?' Then it compares that output with what is already mounted. This is the core abstraction boundary.

Step	What triggers it	Main question answered
Schedule update	State setter, parent re-render, context change, or root render.	Which subtree may need work?
Render	React calls affected components.	What should the next element tree look like?

Reconcile	React compares old and new descriptions.	Which parts can be reused, and which must change?
Commit	React applies mutations and effect transitions.	How does the actual DOM now become consistent?
Paint	The browser renders the committed DOM and style state.	What does the user finally see?

This staged model explains why re-render is not the same thing as repaint. A component function can run and return the same visible output. React can then commit little or nothing. Even when React commits, the browser still decides when the new DOM state becomes painted pixels.

9.1 Effects Belong After Commit

Effects such as subscriptions, timers, network synchronization, and direct DOM integrations belong after React commits because those operations are about synchronizing with the outside world. They are not part of describing what the next UI should be.

- Render computes the description.
- Commit updates the DOM.
- Effects run after commit because they depend on the committed result or coordinate with external systems.

9.2 The Derived Data Trap

A frequent React mistake is storing values in state that can be derived directly from other state and props during render. Every redundant state value creates another synchronization problem. If value B is always computed from value A, prefer computing B during render unless computation is truly expensive and measured.

Re-Render Debug Rule

When a React screen looks wrong, ask these questions in order: what state changed, which component re-ran, what JSX did that render produce, what identity changed because of keys or type, and what DOM mutation was committed?

Exercise - Key Stability

Create a list of editable rows. First use array indexes as keys. Then reorder the list and observe state or input value confusion.

Algorithmic direction: Identity bugs are easiest to understand when the UI contains local state such as text inputs. That is where incorrect preservation becomes visible immediately.

10. File Structure and Execution Flow

Tooling varies, but the execution flow is stable. There is an HTML document with a root container. There is an entry JavaScript module that boots React. There is an application component tree whose output becomes DOM nodes under that root.

10.1 Minimal Boot Path

```html
<!-- index.html -->
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

```javascript
// main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

```javascript
// App.jsx
export default function App() {
  return <h1>Hello, React</h1>;
}
```

The browser first parses HTML and creates the DOM, including the root container. Then it loads and runs the entry module. `createRoot` tells React which DOM node it owns. `render` starts the initial React render, which produces an element tree, reconciles it against an empty mounted tree, and commits the initial DOM nodes under that root.

Layer	Responsibility	Failure if broken
index.html	Provides the root mount point and script loading path.	React has nowhere to mount.
Entry module	Creates the root and starts rendering.	Application code never reaches the DOM.
App and child components	Describe the UI from state and props.	Root exists, but intended UI is wrong or empty.
Browser DOM and paint pipeline	Turns committed nodes and styles into visible pixels.	Logic may be correct while layout or visibility is wrong.

Invariant Across Tooling

Whether you use Vite, Create React App, Next.js client components, or another setup, the local client-side invariant remains: root container -> JavaScript entry -> React root -> component render -> DOM commit -> browser paint.

Exercise - Execution Trace

Open a small React project and trace the first paint from index.html to createRoot to App to actual DOM nodes in the browser.

Algorithmic direction: Write the sequence as a numbered list from memory. If any step feels vague, that is the next concept to solidify.

11. Practical Mini Systems

These mini systems are intentionally small. The point is to practice the causal chain, not to accumulate framework features. Each example should be read as an executable pattern: event enters, state or DOM changes, visible output follows.

11.1 Button -> Adds Element

Mini System Brief

Goal: Add a new list item every time the user clicks a button.
Key idea: A click handler creates a node, configures it, and inserts it into a stable parent.
Hint: Creation in memory is not enough. The visible change begins at insertion.

```javascript
const list = document.getElementById('items');
const button = document.getElementById('add-item');
let nextId = 1;

button.addEventListener('click', () => {
  const item = document.createElement('li');
  item.textContent = `Item ${nextId++}`;
  list.appendChild(item);
});
```

11.2 Button -> Removes Element

Mini System Brief

Goal: Remove whichever list item the user targeted.
Key idea: Event delegation lets one stable parent handle removal for present and future children.
Hint: Do not bind listeners repeatedly inside each add path unless there is a reason. Use a parent boundary when possible.

```javascript
const list = document.getElementById('items');

list.addEventListener('click', (event) => {
  const remove = event.target.closest('[data-remove]');
  if (!remove) return;

  const item = remove.closest('li');
  if (item) item.remove();
});
```

11.3 Input -> Updates UI

Mini System Brief

Goal: Mirror an input value into visible UI in React.

Key idea: The browser reports the latest typed value through the event; React stores it in state and re-renders the UI from that state.

Hint: Treat the input as controlled so the state snapshot remains authoritative.

```react
function Mirror() {
  const [text, setText] = useState('');

  return (
    <>
      <input
        value={text}
        onChange={(event) => setText(event.target.value)}
      />
      <p>Preview: {text || 'Nothing typed yet'}</p>
    </>
  );
}
```

11.4 Timer -> Updates UI Over Time

Mini System Brief

Goal: Show a ticking counter in React.

Key idea: A timer triggers future state updates. Cleanup stops the timer when the component goes away.

Hint: The challenge is not incrementing. The challenge is owning timer lifetime correctly.

```react
function Clock() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setSeconds((s) => s + 1);
    }, 1000);

    return () => clearInterval(id);
  }, []);

  return <p>Elapsed: {seconds}s</p>;
}
```

How To Use These Mini Systems

Do not only read them. Rebuild each example from memory, then mutate one dimension: add validation, add deletion, add filtering, add async delay, or add list identity. Mastery comes from controlled variation.

12. Common Mistakes and Correction Patterns

Mistake	Mechanical reality	Correction
Changing a variable and expecting the screen to update automatically	Variables do not repaint anything on their own. Only DOM mutations or React state-driven re-render paths can affect visible UI.	Define a render path. In plain JS, call render after state changes. In React, store UI-driving data in state.
Using innerHTML as the default update tool	Replacing markup recreates descendant nodes and can destroy listeners or local DOM state.	Prefer direct node operations or declarative rendering through React.
Mutating React state objects or arrays in place	Reference identity may not change, and even when React re-renders, mutated shared objects create confusing bugs.	Create new arrays or objects when updating state.
Reading the latest React state from a stale callback	Callbacks close over the render in which they were created.	Use functional updates or refs depending on whether the value should trigger re-render.
Using list index as key in reorderable or editable lists	Index-based identity shifts when order changes, so state and DOM continuity map to the wrong logical item.	Use stable domain identity such as item.id.
Directly manipulating DOM inside a React-owned subtree	React's next commit may overwrite or discard that manual mutation because React owns the description.	Either let React describe that change or isolate imperative integrations carefully with refs and effects.
Forgetting to clean up timers or listeners	Background callbacks keep firing even after the related UI no longer exists.	Return cleanup from effects and remove listeners when the ownership boundary ends.
Storing derived data as separate state by default	Redundant state introduces synchronization bugs because now multiple values must stay consistent.	Compute derived values during render unless there is a measured performance reason not to.

12.1 Debugging Questions That Matter

1. What is the source of truth for the value that should be visible right now?
2. Which event or timer changes that source of truth?
3. Which render path converts that source of truth into visible UI?
4. Who owns this DOM node: my imperative code, or React?
5. What gives this list item or component stable identity across updates?
6. What background process must be cleaned up when this UI disappears?

Final Diagnostic Principle

Most UI bugs are not random. They are ownership bugs, identity bugs, timing bugs, or stale data bugs. If you classify the bug correctly, the fix usually becomes obvious.

13. Operational Checklist For Real Projects

- Keep a single clear source of truth for each piece of visible state.
- Prefer deriving UI from data over scattering manual mutations across handlers.
- Use event delegation when children are dynamic in plain DOM code.
- Treat keys as identity, not as a linter tax.
- Use effects for synchronization with external systems, not for routine value derivation.
- Clean up timers, subscriptions, and listeners as part of the feature, not as an afterthought.
- When the screen is wrong, trace the full chain: event -> state -> render -> DOM commit -> paint.

If you can perform that trace without hand-waving, you are no longer operating at tutorial level. You are reasoning about UI as a system. That is the threshold that makes unfamiliar frameworks, libraries, and production codebases much easier to enter.

End of final manual - JavaScript and React

Build small systems, vary them deliberately, and keep tracing cause and effect.